

Bubble sort

ตัวอย่างการทำงาน สมมติว่าเราเรามีลิสต์ [5, 1, 4, 2, 8]

หลักการคือ "หยิบตัวเลขที่ละคู่ที่อยู่ติดกัน" วิ่งจากซ้ายไปขวา

รอบที่ 1: [5, 1, 4, 2, 8]

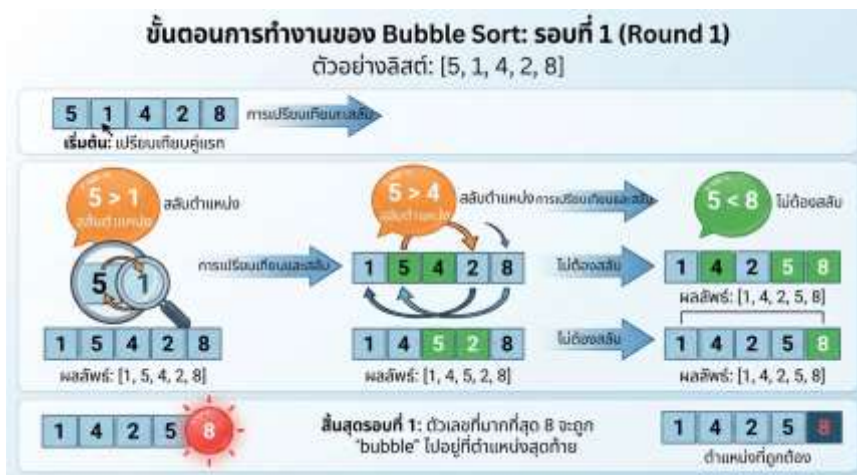
เปรียบเทียบ 5 กับ 1: เนื่องจาก $5 > 1$ ให้สลับตำแหน่งกัน จะได้ [1, 5, 4, 2, 8]

เปรียบเทียบ 5 กับ 4: เนื่องจาก $5 > 4$ ให้สลับตำแหน่งกัน จะได้ [1, 4, 5, 2, 8]

เปรียบเทียบ 5 กับ 2: เนื่องจาก $5 > 2$ ให้สลับตำแหน่งกัน จะได้ [1, 4, 2, 5, 8]

เปรียบเทียบ 5 กับ 8: เนื่องจาก $5 < 8$ ไม่ต้องสลับตำแหน่ง จะได้ [1, 4, 2, 5, 8] เหมือนเดิม สิ้นสุดรอบ

ที่ 1 ตัวเลขที่มากที่สุดคือ 8 จะถูก "bubble" ไปอยู่ที่ตำแหน่งสุดท้ายของรายการแล้ว



รอบที่ 2: [1, 4, 2, 5, 8]

เปรียบเทียบ 1 กับ 4: $1 < 4$ ไม่สลับ

เปรียบเทียบ 4 กับ 2: $4 > 2$ สลับ จะได้ [1, 2, 4, 5, 8]

เปรียบเทียบ 4 กับ 5: $4 < 5$ ไม่สลับ

เปรียบเทียบ 5 กับ 8: $5 < 8$ ไม่สลับ (ไม่จำเป็นต้องเปรียบเทียบกับ 8 แล้ว เพราะ 8 อยู่ในตำแหน่งที่ถูกต้องแล้ว)



รอบที่ 3:

เปรียบเทียบ 1 กับ 2: $1 < 2$ ไม่สลับ

เปรียบเทียบ 2 กับ 4: $2 < 4$ ไม่สลับ

เปรียบเทียบ 4 กับ 5: $4 < 5$ ไม่สลับ

สิ้นสุดรอบที่ 3 จะไม่มีการสลับเกิดขึ้นเลย ซึ่งเป็นสัญญาณบ่งบอกว่ารายการถูกเรียงลำดับเสร็จสมบูรณ์แล้ว

แต่โปรแกรมยังทำงานต่อไป ด้วย `for i in range (n-1):`

แปลว่า สำหรับโจทย์ข้อนี้ โปรแกรมทำงาน 4 รอบ เพราะข้อมูลมี 5 ตัว แต่รอบที่ 3 4 ไม่มีการสลับ

การ "หยิบตัวเลขที่อยู่ติดกันมาเปรียบเทียบและสลับที่กันไปเรื่อย ๆ จากซ้ายไปขวา" เป็น

หัวใจสำคัญที่สุดของ Bubble Sort

สูตรคำนวณจำนวนรอบของการทำงาน ก็คือ $n-1$ นั่นเอง

ข้อมูล `[5, 1, 4, 2, 8]` มี 5 ตัว จากโปรแกรม bubble sort ที่เรียน ตอน ทำงาน 4 รอบ

แล้วมีสูตรคำนวณจำนวนสลับไหม คำตอบคือ ไม่มีสูตรตายตัว ขึ้นอยู่กับข้อมูล

แต่มีแนวทางคำนวณนะครับ คือ จำนวนครั้งในการสลับทั้งหมด จะเท่ากับ "จำนวนคู่อันดับที่เรียงผิดลำดับ" ในลิสต์เริ่มต้นเสมอ

ตัวอย่างจากลิสต์ `[5, 1, 4, 2, 8]`:

ลองนับคู่ที่ตัวหน้ามากกว่าตัวหลัง ($i < j$ แต่ $arr[i] > arr[j]$):

1. คู่ของ 5: (5, 1), (5, 4), (5, 2) $\rightarrow$ 3 คู่

2. คู่ของ 1: ไม่มี

3. คู่ของ 4: (4, 2) $\rightarrow$ 1 คู่

4. คู่ของ 2: ไม่มี

5. คู่ของ 8: ไม่มี

- รวมจำนวนคู่ที่วางผิดลำดับ : $3 + 1 = 4$

- สรุป: ไม่ว่ารอบจะรัน bubble sort ก็รอบ ลิสต์นี้จะเกิดการสลับทั้งหมด 4 ครั้งแน่นอน

คำว่า Inversions แปลว่า "จำนวนคู่ที่วางผิดลำดับ" คำว่า Swap แปลว่า "จำนวนครั้งที่ต้องสลับ"

ใน Bubble Sort การ Swap 1 ครั้ง จะช่วยลด Inversion ไปได้ 1 คู่เสมอ

จำนวน Inversions ทั้งหมดที่มีอยู่ในลิสต์ตั้งแต่แรก จึง เท่ากับ จำนวนครั้งที่เราต้อง Swap จนกว่าจะเรียงเสร็จพอดี

Name **Antigravity Pair Programmer** id **66020416-DSA**

Given the array [64, 34, 25, 12, 22, 11, 90], trace the execution of the Bubble Sort program **that we learned in class to** sort the array in ascending order.

- How many **passes** (outer loop iterations) will the algorithm make?

Answer : **6 passes** (สำหรับ $n = 7$ ตัว ลูปนอกจะวน $n - 1 = 7 - 1 = 6$ รอบ)

- How many total swaps from start to finish?

Answer : **14 total swaps** (เท่ากับผลรวม **Inversions** = $5 + 4 + 3 + 1 + 1 = 14$ ครั้ง)

- Complete the step-by-step trace of Pass 1.

Initial: [64, 34, 25, 12, 22, 11, 90]

- Step 4: Compare (64, 22) → **[34, 25, 12, 22, 64, 11, 90]**]

- Show the array state after **Pass 1**.

Initial Array: [64, 34, 25, 12, 22, 11, 90]

Pass 1 Result: [**[34, 25, 12, 22, 11, 64, 90]**]

- How many total swaps after Pass1?

Answer : **5 swaps** (สลับ 5 คู่แรก: (64,34), (64,25), (64,12), (64,22), (64,11) และคู่สุดท้าย (64,90) ไม่สลับ)

[ตารางสรุป Pass-by-Pass และหลักการคำนวณ Swaps/Inversions ครบถ้วน]:

- * สรุปผลการทำงานของ Bubble Sort บน [64, 34, 25, 12, 22, 11, 90] ($n = 7$):
- * Pass 1: เกิด 5 swaps -> ได้ [34, 25, 12, 22, 11, 64, 90] (90 อยู่ตำแหน่งสุดท้าย)
- * Pass 2: เกิด 4 swaps -> ได้ [25, 12, 22, 11, 34, 64, 90] (64 อยู่ตำแหน่งรองสุดท้าย)
- * Pass 3: เกิด 3 swaps -> ได้ [12, 22, 11, 25, 34, 64, 90] (34 เข้าที่)
- * Pass 4: เกิด 1 swap -> ได้ [12, 11, 22, 25, 34, 64, 90] (25 เข้าที่)
- * Pass 5: เกิด 1 swap -> ได้ [11, 12, 22, 25, 34, 64, 90] (22 เข้าที่)
- * Pass 6: เกิด 0 swaps -> ได้ [11, 12, 22, 25, 34, 64, 90] (ตรวจเช็คครบ $n-1$ รอบ สมบูรณ์)
- * จำนวน Total Swaps ทั้งหมด = $5 + 4 + 3 + 1 + 1 + 0 = 14$ ครั้ง (ตรงกับจำนวน Inversions เป๊ะ)
- * ข้อควรระวังข้อสอบ: ลูปนอกวน $n - 1 = 6$ รอบเท่านั้น ห้ามตอบ 7 รอบเด็ดขาด!

สมมติว่าเรามีลิสต์ [64, 34, 25, 12, 22, 11, 90]

รอบที่ 1: [64, 34, 25, 12, 22, 11, 90]

(64, 34) -> สลับ -> [34, 64, 25, 12, 22, 11, 90]

(64, 25) -> สลับ -> [34, 25, 64, 12, 22, 11, 90]

(64, 12) -> สลับ -> [34, 25, 12, 64, 22, 11, 90]

(64, 22) -> สลับ -> [34, 25, 12, 22, 64, 11, 90]

(64, 11) -> สลับ -> [34, 25, 12, 22, 11, 64, 90]

(64, 90) -> ไม่สลับ -> [34, 25, 12, 22, 11, 64, 90]

หลังรอบที่ 1: 90 (ตัวที่ใหญ่ที่สุด) จะอยู่ที่ตำแหน่งสุดท้าย

รอบที่ 2: [34, 25, 12, 22, 11, 64, 90] (พิจารณาถึงตำแหน่งก่อน 90 เพราะ 90 มีค่ามากที่สุด อยู่ขวาสุดแล้ว)

(34, 25) -> สลับ -> [25, 34, 12, 22, 11, 64, 90]

(34, 12) -> สลับ -> [25, 12, 34, 22, 11, 64, 90]

(34, 22) -> สลับ -> [25, 12, 22, 34, 11, 64, 90]

(34, 11) -> สลับ -> [25, 12, 22, 11, 34, 64, 90]

(34, 64) -> ไม่สลับ -> [25, 12, 22, 11, 34, 64, 90]

หลังรอบที่ 2: 64 จะอยู่ที่ตำแหน่งรองสุดท้าย กระบวนการนี้จะดำเนินต่อไป ด้วย **for i in range(n):**